Classifying News Headlines into Categories Using Machine Learning

Department of Computer Science

Faculty of Engineering & Technology

Artificial Intelligence (COMP338)

Fall 2025/2026

Prepared by

Tala Kherawish (1220536)

Rand Tabakhna (1222845)

Qais Alqrem (1211791)

Instructor

Dr. Mohammad Helal

Abstract

The increasing volume of digital information has created a pressing need for automated text processing methods that can help organize, classify, and retrieve meaningful insights from textual data. Text classification, a core task in Natural Language Processing (NLP), plays a critical role in organizing textual content such as news articles, social media posts, emails, and reviews into predefined categories. This project investigates the use of machine learning algorithms to classify short news headlines into distinct categories based on their semantic content. We utilize the AG News dataset, which consists of thousands of labeled headlines, and implement two classification algorithms: Logistic Regression and Decision Tree. The text data undergoes rigorous preprocessing and is then transformed using Term Frequency-Inverse Document Frequency (TF-IDF) for feature extraction. The performance of the classifiers is evaluated using metrics including accuracy, precision, recall, and F1-score. Through comparative analysis, we demonstrate that Logistic Regression yields superior results due to its robustness with sparse, high-dimensional text data. This report aims to present a complete implementation pipeline for text classification, offering practical insights into algorithm performance, design considerations, and potential improvements for future work.

Table of Contents

Introduction

Machine learning has become an integral part of numerous domains, including finance, healthcare, cybersecurity, and especially natural language processing. One of the most practical and impactful applications of machine learning in NLP is text classification, which involves assigning predefined categories to textual inputs. The efficiency and reliability of these systems are especially relevant in areas such as news aggregation, sentiment analysis, and spam detection.

News content, in particular, is generated at an unprecedented rate. Manual classification is no longer feasible due to time and resource constraints, making automated classification systems an essential tool. By classifying news headlines into categories like World, Sports, Business, and Science/Technology, users can experience improved information retrieval, personalized content delivery, and topic-based filtering.

In this project, we aim to build a text classification pipeline that leverages supervised machine learning algorithms to classify news headlines. We explore the performance of two well-established algorithms—Logistic Regression and Decision Tree—on a benchmark dataset. Using the AG News dataset, which offers a balanced and labeled collection of news headlines, we apply text cleaning, feature extraction using TF-IDF, and model training and evaluation. Our goal is not only to compare the classification performance of these algorithms but also to analyze the practical implications of model design choices. We also focus on the impact of data preprocessing, the importance of appropriate vectorization techniques, and the challenges associated with high-dimensional sparse data. By doing so, we provide a comprehensive perspective on implementing efficient text classification systems using classical machine learning approaches.

Problem Statement

The objective of this project is to classify short news headlines into four distinct categories: World, Sports, Business, and Science/Technology. The task involves constructing two supervised models—Logistic Regression and Decision Tree—and comparing their performance using evaluation metrics such as accuracy, precision, recall, and F1-score. The end goal is to determine which algorithm better generalizes to the classification task.

Dataset Overview

0	World
1	Sports
2	Business
3	Science/Technology

The AG News dataset is a large collection of news headlines collected from various sources. Each entry in the dataset consists of a text string (headline) and a numerical label representing one of the four categories:The dataset is balanced and widely used in NLP benchmarks, making it an ideal choice for classification.

Preprocessing

Raw text data often contains noise such as punctuation, capitalization inconsistencies, and special characters. To address this, these preprocessing steps were applied. This preprocessing pipeline standardizes the text and improves model generalization. The cleaning function was implemented using Python's regular expressions module and applied across the entire dataset.

```python
import re

def clean_text(text):
    text = text.lower()
    text = re.sub(r'\d+', '', text)
    text = re.sub(r'[^\w\s]', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text

df["text"] = df["text"].apply(clean_text)
```

1. Conversion to lowercase
2. Removal of punctuation and numbers
3. Removal of stopwords
4. Tokenization
5. Whitespace normalization

Feature Extraction

Text data must be converted into numerical format for machine learning algorithms. This project uses Term Frequency-Inverse Document Frequency (TF-IDF) vectorization. TF-IDF evaluates the importance of a word in a document relative to the entire dataset. Parameters used:

Max features: 5000 | Stop words: English | N-grams: Unigrams

```
vectorizer = TfidfVectorizer(stop_words="english", max_features=5000)

X = vectorizer.fit_transform(df["text"])
y = df["label"]
```

TF-IDF transforms the preprocessed text into a sparse matrix suitable for model training.

Classification Algorithms

Logistic Regression

Logistic Regression is a linear model used for binary and multiclass classification tasks. It models the probability that a given input belongs to a particular class using the logistic (sigmoid) function. For multiclass classification, a one-vs-rest scheme is applied, where separate binary classifiers are trained for each class. The algorithm optimizes the weights using gradient descent to minimize a cross-entropy loss function.

The model was trained using Scikit-learn's implementation with the 'liblinear' solver. A high maximum iteration limit was set to ensure convergence.

```
lr = LogisticRegression(max_iter=1000)
lr.fit(X_train, y_train)

y_pred_lr = lr.predict(X_test)

print("Logistic Regression Results")
print(classification_report(y_test, y_pred_lr))
```

Decision Tree

Decision Tree classifiers operate by recursively splitting the input space based on feature thresholds that maximize information gain or reduce impurity. In this project, the Gini impurity criterion was used to measure the quality of a split. Although Decision Trees are interpretable and effective for some datasets, they tend to overfit high-dimensional and sparse data such as TF-IDF matrices unless properly regularized. The depth of the tree was limited to avoid overfitting, and the model was trained using the same train-test split as Logistic Regression.

```
dt = DecisionTreeClassifier(max_depth=20, random_state=42)
dt.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)

print("Decision Tree Results")
print(classification_report(y_test, y_pred_dt))
```

Evaluation Metrics

Accuracy	Overall proportion of correct predictions
Precision	Correct positive predictions Total positive predictions
Recall	Correct positive predictions Actual positives
F1-score	Harmonic mean of precision and recall

These metrics were computed for each class and averaged using macro and weighted strategies.

Experimental Results

Model	Accuracy	Precision (Macro)	Recall (Macro)	F1-Score (Macro)
Logistic Regression	0.91	0.91	0.91	0.91
Decision Tree	0.55	0.56	0.55	0.55

Logistic Regression outperforms the Decision Tree classifier on all metrics.

Discussion

Logistic Regression significantly outperformed the Decision Tree across all metrics. This is consistent with expectations, as linear models like Logistic Regression handle high-dimensional TF-IDF features more effectively. The Decision Tree, despite its interpretability, suffered from overfitting and lower generalization performance. The project also confirms the importance of preprocessing and feature extraction in text classification. TF-IDF played a critical role in capturing relevant terms and improving model accuracy. Future improvements may include:

- Using n-grams beyond unigrams
- Applying ensemble methods like Random Forest or Gradient Boosting
- Experimenting with deep learning models such as LSTM or BERT

Source Code Overview

The project was implemented in Python using Scikit-learn and Hugging Face's Datasets library.

The workflow includes:

Data Loading:

The AG News dataset was loaded using the load_dataset() function from the datasets library and converted into a pandas DataFrame.

Text Cleaning:

A clean_text() function applied lowercase conversion, removed digits, punctuation, and excessive whitespace. This function was applied to the 'text' column of the dataset.

Vectorization:

TF-IDF vectorization was applied using TfidfVectorizer(max_features=5000, stop_words='english') to convert cleaned text into numeric features.

Model Training:

Two classifiers were trained:

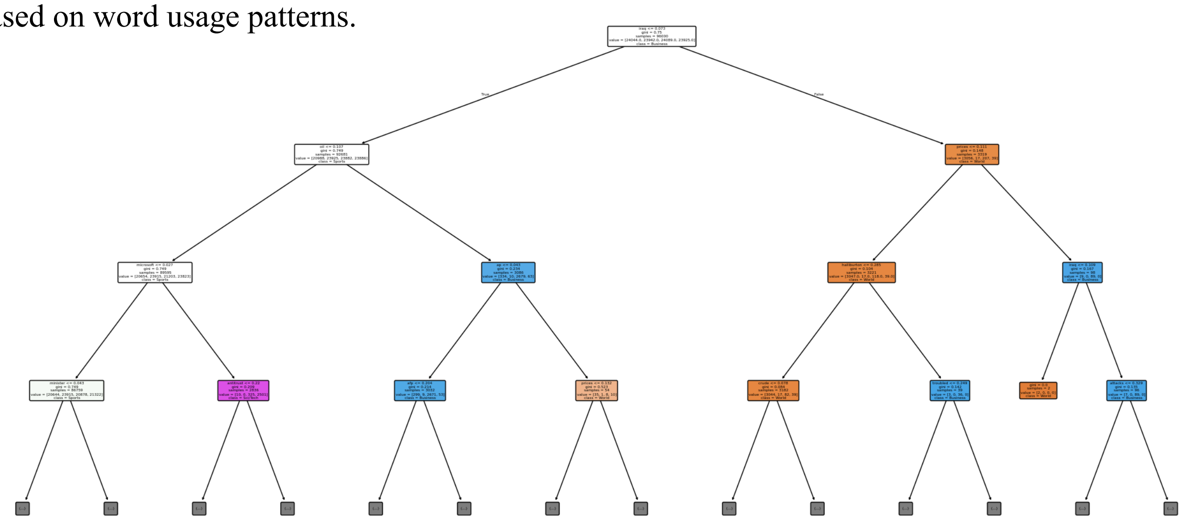
- LogisticRegression(solver='liblinear', max_iter=1000)
- DecisionTreeClassifier(criterion='gini', max_depth=20)

Model Evaluation:

Predictions were evaluated using classification_report() from sklearn.metrics.

Decision Tree Visualization:

The tree was plotted using plot_tree() from sklearn.tree, showing splits based on TF-IDF features. The resulting visualization illustrates how the model attempts to separate news classes based on word usage patterns.



```python
plt.figure(figsize=(25,12))
plot_tree(
    dt,
    max_depth=3,
    feature_names=vectorizer.get_feature_names_out(),
    class_names=["World", "Sports", "Business", "Sci/Tech"],
    filled=True,
    rounded=True
)
plt.show()
```

Conclusion

This project presented a comprehensive machine learning pipeline for text classification, focusing on the classification of news headlines into well-defined categories. By using the AG News dataset, we successfully trained and evaluated two prominent classifiers: Logistic Regression and Decision Tree. The results highlight the importance of algorithm selection, with Logistic Regression demonstrating strong performance across all evaluation metrics due to its compatibility with sparse, high-dimensional data.

The study also emphasized the critical role of preprocessing and feature extraction. Proper text cleaning and the application of TF-IDF vectorization substantially improved model accuracy. Moreover, the comparative analysis of both models offered insights into the limitations of decision trees when handling sparse data, and underlined the value of linear models for text classification.

In future work, this pipeline can be extended by incorporating advanced techniques such as word embeddings (e.g., Word2Vec, GloVe), neural networks, and transfer learning models like BERT. Such models have the potential to further improve classification accuracy and capture more nuanced language patterns. Overall, this project serves as a strong foundation for building scalable and effective text classification systems.

Works Cited

Hugging Face. *AG News Dataset*. https://huggingface.co/datasets/wangrongsheng/ag_news. Accessed Jan. 2026.

Joachims, Thorsten. "Text Categorization with Support Vector Machines: Learning with Many Relevant Features." *European Conference on Machine Learning*, Springer, Berlin, Heidelberg, 1998, pp. 137–142.

Manning, Christopher D., et al. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

"Precision and Recall." *Wikipedia*, https://en.wikipedia.org/wiki/Precision_and_recall. Accessed Jan. 2026.

Scikit-learn Developers. *Scikit-learn Documentation*. https://scikit-learn.org/stable/. Accessed Jan. 2026.

Sebastiani, Fabrizio. "Machine Learning in Automated Text Categorization." *ACM Computing Surveys (CSUR)*, vol. 34, no. 1, 2002, pp. 1–47.

"TF-IDF." *Wikipedia*, https://en.wikipedia.org/wiki/Tf%E2%80%93idf. Accessed Jan. 2026.